

RSAS: RNA Structure Analysis Suite

v4.0

A desktop app for finding RNA thermometers, riboswitches, and other regulatory structures in bacterial sequences.

You give it a pool of candidate sequences. It folds each one at the temperatures you care about, finds the regulatory hairpin and the ribosome binding site, measures how buried that site is as the temperature changes, and ranks the candidates so the interesting ones float to the top. No scripting, no command line if you don't want one.

Formerly RNA Thermometer Finder.

The 60-second version

1. Install ViennaRNA (`brew install viennarna`), then run from source: `pip install -r requirements.txt && python main.py`. Or grab a pre-built app when a release has one attached.
2. Drag a FASTA file onto the window.
3. Click **Analyze**. RSAS folds every sequence at 25, 37, and 42 °C (change these to whatever your biology needs), locates the hairpin and the RBS, and works out how paired the RBS is at each temperature.
4. Click **Export**. You get an Excel workbook and a CSV, with the best thermometer candidates sorted to the top.

Don't have a file handy? Load `Examples/Test_Thermo_RV.fasta` and watch the whole thing run.

A quick note on what RSAS is *for*. The signal it's built to surface is a sequence whose RBS is locked up in a hairpin at low temperature and exposed at high temperature, the classic RNA thermometer. The temperature-difference columns are where you look for that. Everything else exists to give those numbers context.

What it does

RSAS reads RNA sequences (FASTA, CSV/TSV, two-column text, or GenBank), folds them at the temperatures you set using ViennaRNA, and reports the structural features that matter for post-transcriptional regulation. For each sequence it runs roughly this pipeline:

1. Fold the full-length sequence at every temperature (minimum free energy, and the partition function if you ask for it).
2. Find the regulatory hairpin, either the terminal stem-loop or the one that sequesters the Shine-Dalgarno sequence.
3. Fold that hairpin on its own at each temperature.
4. Measure base composition (AU%, GC%, GU%) for both the full sequence and the hairpin.
5. Locate the RBS and compute what fraction of it is paired at each temperature.
6. If you've turned on motif search, find every occurrence of your motif and measure how sequestered each one is.

Results land as a CSV and an Excel workbook with up to three tabs: Full Sequence, Hairpin Analysis, and (when there are motif hits) Motif Matches.

Features

The core analysis

- **Hairpin detection, two ways.** Terminal mode takes the rightmost stem-loop, which works when the regulatory element sits at the 3' end of the leader. RBS-based mode instead finds the hairpin that buries the Shine-Dalgarno, with an AUG fallback for fourU-style thermometers where the RBS overlaps the start codon.
- **Your temperatures, not ours.** Configure anywhere from 1 to 5 folding temperatures. The defaults are 25/37/42, but if you study cold-shock elements you'll want something like 15/25/37. Every column, key, and Excel header re-derives itself from whatever you pick.
- **MFE folding** at each temperature via ViennaRNA, the single most stable structure as a dot-bracket string.
- **Partition function** (optional): ensemble free energy, mean pairing probability, and per-nucleotide unpaired probabilities. More informative than MFE, and the slowest thing in the pipeline, so leave it off when you don't need it.
- **RBS sequestering** with a configurable anchor and window. By default RSAS looks 5 to 13 nucleotides upstream of the last AUG, but the RBS Window tab lets you change the anchor (it's IUPAC-aware, so `DTG` catches all three bacterial starts AUG/GUG/UUG), choose first or last match, and widen or narrow the spacing. Leave it alone and you get the classic behavior, byte for byte.
- **Composition analysis** for the full sequence and the extracted hairpin, with optional "in range" filters so you can flag sequences that fall outside your target window.

Motif and sequence finder

Search every sequence for a nucleotide pattern written in IUPAC degenerate codes (R, Y, S, W, K, M, B, D, H, V, N). It reports *all* overlapping matches, not just the best one, and for each hit gives you the paired percentage, the dot-bracket structure, and the partition-function accessibility at each temperature. There's a semicolon-separated summary in the main CSV and a dedicated Motif Matches tab in the Excel file with one row per hit. The temperature-difference columns are the point: they tell you which motifs open up as things heat up.

Synthetic pool generator

Build random RNA pools from a template of segments, random stretches plus fixed IUPAC motifs. There's a built-in RBS + AUG layout (84 random, then GGAGG, then 8 random, then AUG). You can filter on GC%, AU%, and GU% with independent targets, set the pool size, and fix a random seed when you need the same library twice. Output is FASTA.

Quality scoring

Two scores, both 0 to 6 by default. The Terminal Hairpin Quality Score counts how many hairpin criteria (MFE and composition at each temperature) land inside your configured ranges; the Full-Length Quality Score does the same for the whole sequence. The Quality Score Builder lets you decide which criteria count and how much each one weighs. Results come out sorted by score, best first.

Upstream sequence extraction

Pull the region upstream (or downstream) of a genomic feature straight from local genome and annotation files, or fetch it from NCBI by accession. You control the region length, the direction, and whether the start codon comes along. The output is a FASTA you can load straight into the Analyze page.

Structural motif search (RNArobo)

Search for *structural* motifs, patterns defined by paired helices and loops rather than a flat string, using the bundled RNArobo 2.1.0 engine. You build a descriptor from helices (`h`), single-stranded regions (`s`), and relational elements (`r`), each with its own tolerance for mismatches, mispairs, and insertions. There's a preset list and an interactive builder. The `rnarobo` binary ships for macOS; on Linux, download the authors' ready-made executable into `bin/linux/` (see Optional: RNArobo and Knotty).

Pseudoknot prediction (Knotty)

Predict structures that include pseudoknots, the crossing base pairs that ordinary MFE folding can't represent, using the bundled Knotty engine and its DP09 energy model. You get, per sequence, whether a pseudoknot is present, the predicted structure, and the free energy. From Jabbari et al.

(2018), *Bioinformatics* 34(22):3849-3856. Same binary caveat as RNArobo: macOS is bundled; on Linux, build it into `bin/linux/` (see Optional: RNArobo and Knotty).

Structure deck (RNAfold Deck)

Turn the hairpins of one or more quality tiers into a PowerPoint. For each hairpin it folds the full-length sequence, the hairpin alone, or both, at one to five temperatures, and draws the MFE structure the way the RNAfold webserver does (naview layout, nucleotides colored by base-pair probability), with the sequence, MFE and dot-bracket beside it. Folding and drawing are local, with ViennaRNA (`RNAfold -p -d2 --noLP`, the webserver's defaults), so no internet connection or browser is needed. Every slide is re-read and checked before the deck is saved, and each fold is cross-checked against the RSAS sheet. See RNAfold Deck.

Output and export

- CSV containing only the columns you've enabled in the Output Columns dialog.
- Excel workbook with up to three tabs (Full Sequence, Hairpin Analysis, Motif Matches).
- Presets to get you started: Hairpin Analysis, Full Sequence Analysis, Riboswitch, Full Export. Save your own, too.
- Timestamped filename suggestions so you don't overwrite yesterday's run.

The interface

A CustomTkinter desktop UI with a System / Light / Dark switch in the header (it follows your OS theme by default). The sidebar carries nine pages: Analyze, Results, Settings, Sequence Extractor, Synthetic Pool, RNArobo Search, Pseudoknot Finder, RNAfold Deck, and About (version, how to cite, contacts). The Results page has its own Export button. Drag and drop files onto the window, drive it from the keyboard (Cmd/Ctrl+O to open, +R to run, +E to export), and spread large runs across multiple CPU cores. Progress shows up in the log with toast notifications.

Quick start

```
git clone https://github.com/<owner>/RSAS.git && cd RSAS
conda env create -f environment.yml        # Windows: environment-windows.yml
conda activate rsas
python main.py                             # desktop application
python -m RnaThermofinder analyze Examples/sample_dataset.fasta   # or the command line
```

Conda, Docker and native installation on Windows, macOS and Linux are covered step by step in **INSTALL.md**.

Optional: RNArobo and Knotty

Only the RNArobo Search and Pseudoknot Finder pages need these external programs; everything else works without them. Both are installed **inside your RSAS folder** (`bin/<platform>/`), so no administrator rights are needed. On macOS they are already bundled in `bin/macos/` (run `xattr -dr com.apple.quarantine bin/macos` once; Apple Silicon also needs Rosetta 2). On Linux and WSL2, from your RSAS folder:

```
# RNArobo 2.1.0: ready-made Linux executable from its authors (Comenius University in
Bratislava)
mkdir -p bin/linux
curl -L -o bin/linux/rnarobo https://compbio.fmph.uniba.sk/rnarobo/rnarobo-2.1.0-linux64
chmod +x bin/linux/rnarobo

# Knotty: build from source (needs a C++11 compiler and CMake >= 3.1, e.g. conda install -c
conda-forge cmake)
RSAS="$PWD"
mkdir -p ~/tools && cd ~/tools
curl -L -o Knotty-master.zip https://github.com/HosnaJabbari/Knotty/archive/master.zip
unzip -q Knotty-master.zip && cd Knotty-master
cmake -H. -Bbuild && cmake --build build
cp "$(find . -maxdepth 2 -type f -name knotty -perm -u+x | head -n 1)" "$RSAS/bin/linux/"
cp -R simfold "$RSAS/bin/linux/"            # Knotty's energy parameters, next to the
executable
cd "$RSAS"
```

Building from source on macOS, Windows (WSL2), Docker and how to check the engines: INSTALL.md, section 8.

A first run, start to finish

1. **Load sequences.** Click Browse or drag a FASTA/CSV/TSV onto the window. Try `Examples/Test_Thermo_RV.fasta` if you just want to see output.
2. **Adjust settings if you need to.** Most defaults are sensible. The ones worth checking first: folding temperatures (Analysis Settings) and which columns you want (Output Columns).
3. **Run.** Click Analyze or press Cmd/Ctrl+R. Watch the log; a few dozen short sequences finish in seconds, big files with the partition function on can take a while.
4. **Export.** Cmd/Ctrl+E, pick Excel or CSV, done. Open the Full Sequence tab and sort by quality score if it isn't already.

Output columns

What you get depends on what you've enabled, but the categories are:

Category	Example columns
Sequence info	Name, sequence, structure at each temperature
Full-length MFE	MFE at each temperature
Composition	AU%, GC%, GU%, in-range flags
RBS sequestering	RBS sequence and structure, paired% per temperature, temperature diffs
Hairpin	Detection method, sequence, structure, composition, MFE per temperature
Partition function	Ensemble energy, mean pairing probability, RBS accessibility
Motif finder	Pattern, match count, positions, paired%, structure, accessibility, diffs
Quality scores	Hairpin score, full-length score, class, breakdown

Project structure

```
RSAS/
├── main.py                    # entry point
├── settings_manager.py        # config persistence + output column definitions
├── setup.py                   # package setup
├── build_app.py               # PyInstaller build script
├── requirements.txt
├── RSAS.spec                  # PyInstaller spec
├── INSTALL.md                 # installation guide (Conda, Docker, Windows, macOS, Linux)
├── environment.yml            # Conda environment, Linux/macOS
├── environment-windows.yml    # Conda environment, Windows
├── Dockerfile                 # container image (CLI + optional X11 GUI)
├── bin/                       # bundled engines (rnarobo, knotty) per platform
├── Examples/                  # sample FASTA inputs
├── RnaThermofinder/
│   ├── core/
│   │   ├── FastaParse.py      # FASTA/CSV/TSV input parsing
│   │   ├── HairpinAnalysis.py # the main analysis pipeline
│   │   └── rbs_config.py      # configurable RBS anchor/window
│   ├── gui/
│   │   ├── RNAGUI.py          # main application window
│   │   ├── settings_dialog.py        # analysis + performance settings
│   │   ├── settings_dialog_csv.py    # output column config
│   │   ├── motif_finder_dialog.py    # motif search config
│   │   ├── quality_score_builder.py  # quality score criteria
│   │   ├── synthetic_pool_dialog.py  # pool generator UI
│   │   ├── rnarobo_dialog.py         # structural motif search UI
│   │   ├── knotty_dialog.py          # pseudoknot finder UI
│   │   ├── upstream_extractor_dialog.py
│   │   └── sequence_settings_dialog.py
│   ├── deck/                         # RNAfold Deck: tier hairpins -> structures -> PowerPoint
│   │   ├── local_fold.py             # ViennaRNA fold + webserver-style drawing (SVG/PNG)
│   │   ├── engine.py                 # select, trace, fold, build + verify
│   │   ├── panel.py                  # sidebar page (also a standalone window)
│   │   └── cli.py                    # python -m RnaThermofinder.deck run ...
│   └── utils/
│       ├── analysis_helpers.py       # CSV/Excel output builders
│       ├── motif_finder.py           # IUPAC motif search + sequestering
│       ├── quality_scoring.py        # scoring logic
│       ├── synthetic_pool_generator.py
│       ├── rnarobo_wrapper.py        # RNArobo subprocess + parser
```

```
|       ├── knotty_wrapper.py         # Knotty subprocess + parser
|       └── upstream_extractor.py
├── Data/
|   ├── Inputs/                # optional place to keep inputs (load from anywhere)
|   └── Outputs/               # placeholder; the app actually writes to ~/.rsas/Data/Outputs/
└── docs/
    ├── installation.md
    └── usage.md
```

Building a standalone app

```
pip install pyinstaller
python build_app.py
```

That gives you `dist/RSAS.app` (macOS), `dist/RSAS.exe` (Windows), or `dist/RSAS` (Linux). You can also call PyInstaller on the spec directly:

```
pyinstaller RSAS.spec
```

The build takes a few minutes and bundles Python, the dependencies, and the ViennaRNA bindings, so the result runs on a machine with none of that installed.

Documentation

- **Installation guide** covers Conda, Docker and native installation on Windows, macOS and Linux, the optional RNArobo/Knotty engines, verification, and troubleshooting.
- **Usage guide** walks through the GUI, loading and running, the output formats, the scripting API, and a pile of practical tips.
- **Features and tools, in depth** is one detailed page per feature and tool: every option, what it produces, a worked example, and the gotchas.
- **How it works** is the methods document: the folding model, the RBS and hairpin algorithms, how the quality score is computed, what the thermometer signal is, and where the limits are.
- **Output column reference** defines every CSV/Excel column with its meaning and units.
- **Worked example** runs the bundled sample data end to end with real output numbers and explains how to read them.

Citation

If RSAS shows up in published work:

```
Vaknin, R. (2025). RSAS: RNA Structure Analysis Suite v3.2.
GitHub: https://github.com/RoyCyber1/RSAS
```

The repository also ships a `CITATION.cff`, so GitHub's "Cite this repository" button generates APA and BibTeX entries for you.

Changelog

See CHANGELOG.md for the full version history.

License

RSAS is MIT-licensed (see LICENSE.md) and is non-commercial academic software. It bundles third-party academic tools (RNArobo, Knotty) that carry their own terms; see THIRD-PARTY-NOTICES.md.

Author

Roy Vaknin

- Email: roycyber13@gmail.com
- GitHub: RoyCyber1

Acknowledgments

- ViennaRNA for the folding algorithms RSAS leans on.
- RNArobo for structural motif search.
- Knotty (Jabbari et al. 2018) for pseudoknot prediction.
- Dr. Abdelsayed for experimental validation data.
- The SCREAM team for testing and feedback.